Milestone 2: Leduc Hold'em and the Faster CFR Variants

holdem-cfr walkthrough

Why Leduc

Kuhn poker proves the engine is correct, but it is too small to show differences in speed. Leduc hold'em, from Southey et al. (2005), is the standard next step because it contains every structural feature of real hold'em in miniature. It has private cards, a public card dealt between two betting rounds, bet sizes that grow in the later round, a cap on raises, and showdowns where a pair with the board beats a higher card. The deck has six cards, two suits each of jack, queen and king. Each player antes one chip. Bets are two chips in the first round and four in the second, with at most a bet and one re-raise per round. Player 0 acts first in both rounds.

Our implementation keeps the suits distinct, which gives the standard count of **936 information sets**: 468 per player. The count decomposes cleanly, and the decomposition is a good check of the rules. Each round has six betting histories at which someone must act. In round one, the information set is the history plus your private card, which is one of 6, giving 36. Five first-round histories end with the round closing and the game continuing. Each of those can be followed by six second-round decision histories, observed together with your card and a board card, giving $6 \times 5 = 30$ combinations, so $5 \times 6 \times 30 = 900$. Together that is $36 + 900 = 936$. If the builder produced any other number, the betting logic would be wrong somewhere. The test suite asserts exactly 936, and also checks the rules directly: the raise cap, fold payoffs, pair-beats-high-card, split pots, and the maximum pot of thirteen chips each.

CFR+: three small changes, a large speedup

Tammelin's CFR+ (2014) changes vanilla CFR in three ways.

The first change floors cumulative regrets at zero after every update, so $R \leftarrow \max(R + r, 0)$. In vanilla CFR an action that was terrible early accumulates a large negative regret. Even after it becomes the right play, it has to climb out of that hole before regret matching gives it any probability. Flooring removes the hole, so an action becomes playable again the moment it starts looking good.

The second change weights iteration t's contribution to the average strategy by t, which discounts the poor early iterates. The third is alternating updates, which our engine already uses for everything.

There is a subtle implementation point in the first change. The floor must be applied to the information set's *total* regret for the iteration, summed over every history in the set, not to each history's contribution as it is visited. Flooring partial sums gives a different and theoretically unjustified algorithm. This is the second reason the solver accumulates into the separate `instant` buffer and folds it into the cumulative regrets only after the traversal:

```
fn fold_instant_regrets(&mut self, player: usize) {
    for info in self.tree.infosets.iter().filter(|i| i.player == player) {
        for k in info.range() {
            let updated = self.regrets[k] + self.instant[k];
            self.regrets[k] = match self.variant {
                Variant::CfrPlus => updated.max(0.0),
                _ => updated,
            };
            self.instant[k] = 0.0;
        }
    }
}
```

Discounted and Linear CFR

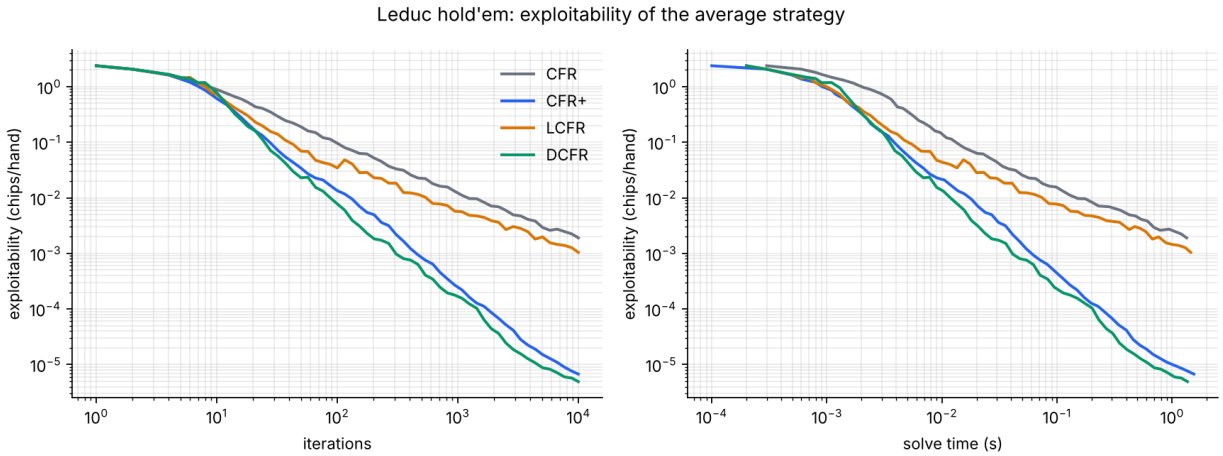
Brown and Sandholm (2019) generalized the idea of forgetting the past. After iteration t, Discounted CFR multiplies positive cumulative regrets by $t^\alpha / (t^\alpha + 1)$ and negative ones by $t^\beta / (t^\beta + 1)$, and multiplies the cumulative strategy by $(t / (t + 1))^\gamma$. With the recommended setting $\alpha = 1.5$, $\beta = 0$, $\gamma = 2$, positive regrets are discounted gently. Negative regrets are halved every iteration, which is a soft version of CFR+'s floor. The average strategy weights iteration t roughly by t^2, so the early iterates fade quickly. Linear CFR is the special case $\alpha = \beta = \gamma = 1$: iteration t's regrets and strategy are both weighted by t, and negative regrets are never floored.

In code, all four variants share one traversal and differ only in the regret update, the weight applied to the average strategy, and the optional discount pass at the end of each iteration. This is why `Variant` is an enum and `DCFR` and `LINEAR` are associated constants rather than separate solvers:

```
pub enum Variant {
    Vanilla,
    CfrPlus,
    Discounted { alpha: f64, beta: f64, gamma: f64 },
}
impl Variant {
    pub const DCFR: Variant = Variant::Discounted { alpha: 1.5, beta: 0.0, gamma: 2.0 };
    pub const LINEAR: Variant = Variant::Discounted { alpha: 1.0, beta: 1.0, gamma: 1.0 };
}
```

Results

Each algorithm ran 10,000 iterations on Leduc, single-threaded, with exact exploitability measured at roughly logarithmically spaced checkpoints. A full iteration over the 9,451-node tree takes about 0.14 ms, so each curve costs about a second and a half.



Leduc exploitability

Algorithm	at 1,000 iterations	at 10,000 iterations
CFR	≈1.2e-2	1.9e-3
Linear CFR	≈5.6e-3	1.0e-3
CFR+	≈2.2e-4	6.7e-6
DCFR (1.5, 0, 2)	≈1.6e-4	4.9e-6

```
CFR   : exploitability 1.9e-3
LCFR  : exploitability 1.0e-3
CFR+  : exploitability 6.7e-6
DCFR  : exploitability 4.9e-6
game value (player 0): -0.0856   (published value: -0.0856)
```

Exploitability is measured in chips per hand. On a log-log plot vanilla CFR has slope about -1, while CFR+ and DCFR fall roughly twice as steeply after the first few dozen iterations. By 10,000 iterations they are almost three hundred times less exploitable than vanilla CFR at the same cost. DCFR is slightly ahead of CFR+ throughout, which matches the Brown and Sandholm paper. The converged game value of -0.0856 for the first player matches the published value for Leduc hold'em. It is the strongest single check we have that the rules, the payoffs and the solver are all right together.

Linear CFR is the instructive outlier. It beats vanilla CFR by only about a factor of two, far less than CFR+ and DCFR. The reason is visible in the parameters. Linear CFR discounts old regrets but never floors or aggressively shrinks *negative* regrets ($\beta = 1$), so actions that were bad early still have to dig out of a deep hole. CFR+

(floor at zero) and DCFR (halve negatives every iteration) both attack exactly that problem, and the speedup comes almost entirely from it. That is a useful lesson for the hold'em agent. Whatever we run at scale should treat negative regrets aggressively.

The milestone gate

The integration tests in `games/tests/leduc.rs` are the gate. They assert the 936 information-set count. They assert that CFR+ reaches exploitability below 2×10^{-4} and a game value within 10^{-3} of -0.0856 after 2,000 iterations. They assert that vanilla CFR's exploitability falls by at least 4× between 100 and 1,000 iterations. And they assert that CFR+ and DCFR are both at least 10× less exploitable than vanilla CFR at 1,000 iterations; the measured ratio is about 50×. A property test runs random iteration counts of every variant and checks the invariants that must always hold. Every average and current strategy sums to one at every information set, every probability lies in [0, 1], every regret is finite, CFR+ regrets are never negative, and exploitability is never negative.

Rust notes from this milestone

The command-line tool uses `clap`'s derive API. You write a plain `enum Command { Solve { .. }, Curves { .. } }` with doc comments, annotate it with `#[derive(Parser)]` and `#[derive(Subcommand)]`, and clap generates the argument parser, the `--help` text and the error messages at compile time. Algorithm names on the command line come from `#[derive(ValueEnum)]` on `enum Algo`, so `--algo cfr-plus` is checked against the real enum and a typo is a parse error, not a silent fallback.

Property tests use `proptest`. A `proptest!` block declares input *strategies*, such as `iters in 1u64..40` or `prop::collection::vec(-1e6f64..1e6, 1..8)`. The framework then generates hundreds of cases. When one fails, it *shrinks* the input to the smallest case that still fails before reporting it, which turns a confusing 7-element failure into a 1-element one.

Reproduce

```
cargo test --workspace
cargo run --release -p cli -- solve --game leduc --algo dcfr --iters 2000
cargo run --release -p cli -- curves --game leduc --iters 10000 --out results/leduc_curves.csv
python3 scripts/plot_curves.py results/leduc_curves.csv results/leduc_exploitability.png "Leduc hold'em"
```

Reading

Tammelin (2014) and Tammelin, Burch, Johanson and Bowling (2015, IJCAI) for CFR+. Brown and Sandholm (2019, AAAI), "Solving Imperfect-Information Games via Discounted Regret Minimization", sections 3 and 4, for DCFR and LCFR and the experiments these curves reproduce in miniature.

Next

Milestone 3 is the 7-card hand evaluator. It is self-contained, it has a brutal exhaustive correctness test over all 133,784,560 seven-card hands, and it is the first place where raw speed, in hands per second, is the deliverable.